

Colecții

Framework-ul Collections

Java Collections Framework (JCF) este un ansamblu de clase și interfețe care oferă structuri de date și algoritmi standardizați pentru manipularea colecțiilor de obiecte.

Caracteristici principale:

- Reutilizabilitate: implementează structuri comune (liste, seturi, map-uri)
- Performanță: implementări optimizate (ex: acces rapid în HashMap)
- Consistență: interfețe comune pentru diferite structuri
- Extensibilitate: poți crea propriile colecții

Ierarhia de bază:

Iterable

|

Collection

|----- List

|----- Set

|----- Queue

Map (separat, nu extinde Collection)

Algoritmi integrați:

Clasa Collections oferă metode utilitare:

- sortare (sort)
- căutare (binarySearch)
- inversare (reverse)
- shuffle

Interfața Collection

Este interfața de bază pentru majoritatea structurilor din JCF.

Metode importante:

- add(E e)
- remove(Object o)
- size()
- isEmpty()
- contains(Object o)
- iterator()

Caracteristici:

- Nu definește ordinea elementelor
- Permite duplicate (depinde de implementare)

Interfața List

Extinde Collection și reprezintă o colecție ordonată.

Caracteristici:

- Elemente indexate (acces prin poziție)
- Permite duplicate
- Menține ordinea inserării

Metode suplimentare:

- get(int index)
- set(int index, E element)
- add(int index, E element)
- remove(int index)

Exemple de utilizare:

- liste de utilizatori
- cozi de task-uri

Clasa ArrayList

Implementare a interfeței List bazată pe array dinamic.

Caracteristici:

- acces rapid $O(1)$
- inserare/ștergere lentă $O(n)$
- redimensionare automată

```
ArrayList<String> lista = new ArrayList<>();  
lista.add("Ana");  
lista.add("Ion");  
  
System.out.println(lista.get(0)); // Ana
```

Dezavantaje:

- cost mare la inserări în mijloc

Clasa LinkedList

Implementare a List bazată pe listă dublu înlănțuită.

Caracteristici:

- inserare/ștergere rapidă $O(1)$
- acces lent $O(n)$
- poate funcționa și ca Queue/Deque

Cazuri de utilizare:

- când ai multe inserări/ștergeri

Exemplu:

```
LinkedList<String> lista = new LinkedList<>();  
lista.add("A");  
lista.addFirst("Start");  
lista.addLast("End");
```

Interfața Set

Extinde Collection și reprezintă o colecție de elemente unice.

Caracteristici:

- nu permite duplicate
- nu garantează ordinea (depinde de implementare)

Cazuri de utilizare:

- eliminare duplicate
- verificare existență rapidă

Clasa HashSet

Implementare a Set bazată pe hash table.

Caracteristici:

- nu păstrează ordinea
- performanță foarte bună $O(1)$
- permite un singur null

Exemplu:

```
HashSet<String> set = new HashSet<>();  
set.add("A");  
set.add("B");  
set.add("A"); // ignorat
```

Clasa LinkedHashSet

Extinde HashSet și păstrează ordinea inserării.

Caracteristici:

- ordine predictibilă
- ușor mai lent decât HashSet

Exemplu:

```
LinkedHashSet<String> set = new LinkedHashSet<>();  
set.add("C");  
set.add("A");  
set.add("B");  
  
// output: C, A, B
```

Clasa TreeSet

Implementare bazată pe arbore roșu-negru.

Caracteristici:

- elementele sunt sortate automat
- nu permite null
- operații $O(\log n)$

Exemplu:

```
TreeSet<Integer> set = new TreeSet<>();  
set.add(5);  
set.add(1);  
set.add(3);  
  
// output: 1, 3, 5
```

Interfața Map

Reprezintă o colecție de perechi cheie-valoare.

Caracteristici:

- cheile sunt unice
- valorile pot fi duplicate

Metode importante:

- put(K key, V value)
- get(K key)
- remove(K key)

- `containsKey(K key)`

Clasa `HashMap`

Implementare a Map bazată pe hash table.

Caracteristici:

- acces rapid $O(1)$
- permite o cheie null
- nu păstrează ordinea

Exemplu:

```
HashMap<String, Integer> map = new HashMap<>();  
map.put("Ana", 25);  
map.put("Ion", 30);  
  
System.out.println(map.get("Ana")); // 25
```

Clasa `TreeMap`

Implementare a Map bazată pe arbore sortat.

Caracteristici:

- cheile sunt ordonate
- operații $O(\log n)$
- nu permite chei null

Exemplu:

```
TreeMap<Integer, String> map = new TreeMap<>();  
map.put(3, "C");  
map.put(1, "A");  
  
// output ordonat: 1=A, 3=C
```

Interfața `Iterator`

Permite parcurgerea colecțiilor într-un mod standard.

Metode:

- hasNext()
- next()
- remove()

Avantaje:

- funcționează uniform pentru toate colecțiile
- permite ștergere sigură în timpul iterării

Exemplu:

```
ArrayList<String> lista = new ArrayList<>();  
lista.add("A");  
lista.add("B");  
  
Iterator<String> it = lista.iterator();  
while (it.hasNext()) {  
    System.out.println(it.next());  
}
```

Diferențe importante sintetizate

Structură	Ordine	Duplicate	Performanță
ArrayList	Da	Da	Acces rapid
LinkedList	Da	Da	Inserare rapidă
HashSet	Nu	Nu	Foarte rapid
LinkedHashSet	Da	Nu	Rapid
TreeSet	Sortat	Nu	Mediu
HashMap	Nu	Chei unice	Foarte rapid
TreeMap	Sortat	Chei unice	Mediu

Exerciții

1. Creează o clasă Student cu attributele:

- nume
- medie

Cerințe:

- a) Creează un ArrayList<Student>
- b) Adaugă cel puțin 5 studenți
- c) Sortează lista descrescător după medie
- d) Afișează studenții
- e) elimină studenții cu media sub 5 folosind Iterator

2. Se dă o listă de numere întregi:

[1, 2, 3, 2, 4, 1, 5, 3]

Cerințe:

- a) Pune valorile într-un ArrayList
- b) Elimină duplicatele
- c) Păstrează ordinea inițială a elementelor

3. Se dă un text:

"ana are mere ana are pere"

Cerințe:

- a) Sparge textul în cuvinte
- b) Folosește un HashMap<String, Integer> pentru a calcula frecvența fiecărui cuvânt
- c) Afișează rezultatul
- d) Afișează cuvintele sortate alfabetic folosind TreeMap

5. Creează o aplicație simplă de tip agendă.

Cerințe:

a) Folosește un `HashMap<String, String>`:

- cheia = nume
- valoarea = număr de telefon

b) Permite:

- adăugare contact
- căutare contact
- ștergere contact

c) Extensie:

d) Folosește `TreeMap` pentru a afișa contactele sortate alfabetic